DS 3 Informatique - 4 décembre : 4h

Ce sujet est composé de trois parties indépendantes.

1 NP-complétude

On rappelle quelques notions de coloration de graphes :

- Une numérotation c d'un graphe $G=(S,A)$ est une fonction de $S \to \mathbb{N}$.
- Une coloration c d'un graphe est une numérotation vérifiant :

$$\forall s,t \in S,\ \ (s,t) \in A \Rightarrow c(s) \neq c(t)$$

- On parle d'une k-coloration s'il s'agit d'une coloration à image dans $[\![0,k-1]\!]$.
- Le nombre chromatique d'un graphe est le plus petit entier k tel que ce graphe admette une k-coloration.

Dans cette partie, on cherche à montrer que le problème général du calcul du nombre chromatique d'un graphe est difficile. Pour ce faire, on étudie plutôt des problèmes de décision liés à la coloration. On définit les problèmes de décision suivants :

- Problème `k-coloration` (pour k fixé)
  - Instance : un graphe G.
  - Question : le graphe G est-il k-colorable?
- Problème `Coloration`
  - Instance : un graphe G et un entier k.
  - Question : le graphe G est-il k-colorable?

Par ailleurs, pour A et B deux problèmes de décision, la notation $A \leqslant^p B$ signifie qu'il existe une réduction polynomiale de A à B.

Question 1 : Montrer que pour tout entier k, `k-coloration` $\leqslant^p$ `Coloration`. On justifiera en particulier le caractère polynomial de la réduction proposé.

Question 2 : Montrer que `2-coloration` $\in$ P. On décrira en français un algorithme polynomial permettant de résoudre le problème.

On représente en C une liste chaînée d'entiers par le type suivant :

```
struct Liste
{
    int val;
    struct Liste* suivant;
};

typedef struct Liste liste;
```

Dès lors, un graphe G sera représenté par un tableau `G` de listes d'adjacence, c'est-à-dire un objet de type `liste**` de telle sorte que si $s \in S$, alors `G[s]` est la liste chaînée des voisins de s. On associera à ce tableau sa taille `n`.

On représentera une numérotation $c : S \to \{0,\dots,k-1\}$ par un tableau `int*` de taille n contenant des éléments de $\{0,\dots,k-1\}$, telle que `c[s]` prend la valeur $c(s)$, pour tout sommet $s \in S$.

Question 3 : Écrire une fonction `bool deux_colorable(liste** G, int n)` qui prend en argument un graphe `G` et son nombre de sommets n et renvoie `true` si G est 2-colorable et `false` sinon. Cette fonction devra avoir une complexité linéaire en le nombre de sommets et d'arêtes de G.

Question 4 : Que pouvez vous dire du problème `1-coloration`?

Les questions suivantes montrent la NP-complétude de `3-coloration`.

Question 5 : Montrer que `Coloration` $\in$ NP.

On rappelle les définitions suivantes : pour un ensemble de variables $V = \{v_1, \ldots, v_n\}$, un *littéral* est une variable ou sa négation, c'est-à-dire de la forme v_i ou $\overline{v_i}$. Une *clause* est une disjonction de littéraux (séparés par des $\vee$). Une formule est dite en *forme normale conjonctive* si c'est une conjonction de clauses (séparées par des $\wedge$). Elle est dite en 3-FNC si de plus chaque clause contient exactement 3 littéraux. Par exemple, la formule φ_0 suivante est en 3-FNC :

$$\varphi_0 = (a \vee b \vee c) \wedge \overbrace{(\overline{a} \vee c \vee \overline{d})}^{\text{clause}} \wedge (b \vee \underbrace{\overline{c}}_{\text{littéral}} \vee d)$$

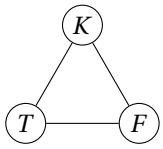
On définit alors le problème de décision :

- Problème `3-SAT`
  - Instance : une formule booléenne φ en 3-FNC.
  - Question : la formule φ est-elle satisfiable ?

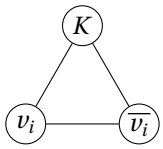
On admet que ce problème est `NP`-complet. La suite de cette partie a pour objectif de montrer qu'il se réduit en temps polynomial au problème `3-coloration`. Pour ce faire, en considérant une formule booléenne φ en 3-FNC, on construit un graphe G_φ qui est 3-colorable si et seulement si φ est satisfiable. L'idée est que deux des couleurs utilisées correspondent à l'affectation *Vrai* ou *Faux* pour chaque littéral, et la troisième couleur est une couleur de contrôle.

Dans le détail, si φ est une formule en 3-FNC sur un ensemble $V = \{v_1, \ldots, v_n\}$ de n variables, alors on définit le graphe $G_\varphi = (S, A)$ par :

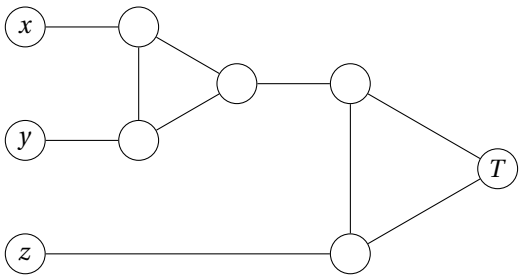
- S contient trois sommets T (*True*), F (*False*) et K (*Control*), un sommet par littéral possible, et 5 sommets supplémentaires par clause ;
- A contient les arêtes suivantes :
  - $\{T, F\}, \{T, K\}, \{F, K\}$



  - pour chaque variable v_i, les arêtes $\{v_i, \overline{v_i}\}, \{v_i, K\}, \{\overline{v_i}, K\}$



  - pour chaque clause $C = x \vee y \vee z$ apparaissant dans φ (où x, y, z sont des littéraux), on rajoute les 10 arêtes telles que décrites dans le graphe ci-dessous, en utilisant les 5 sommets supplémentaires associés à la clause.



Par exemple, la figure 1 représente le graphe G_{φ_0} obtenu pour la formule booléenne φ_0 sur l'ensemble de variables $\{a, b, c, d\}$.

Question 6 : Représenter graphiquement le graphe G_{φ_1} pour $\varphi_1 = (\overline{a} \vee b \vee \overline{c}) \wedge (a \vee b \vee \overline{c}) \wedge (a \vee \overline{b} \vee c)$.

Question 7 : Justifier que la construction de G_φ à partir d'une formule φ en 3-FNC est en temps polynomial en fonction de la taille de φ celle-ci étant son nombre de variables et de clauses.

Question 8 : Montrer que le graphe de la figure 2 est 3-colorable et que pour toute 3-coloration, au moins l'un des sommets x, y ou z a la même couleur que T.

Question 9 : On suppose que φ est satisfiable. Montrer que G_φ est 3-colorable. On expliquera comment construire une 3-coloration de G_φ à partir d'une valuation de φ.

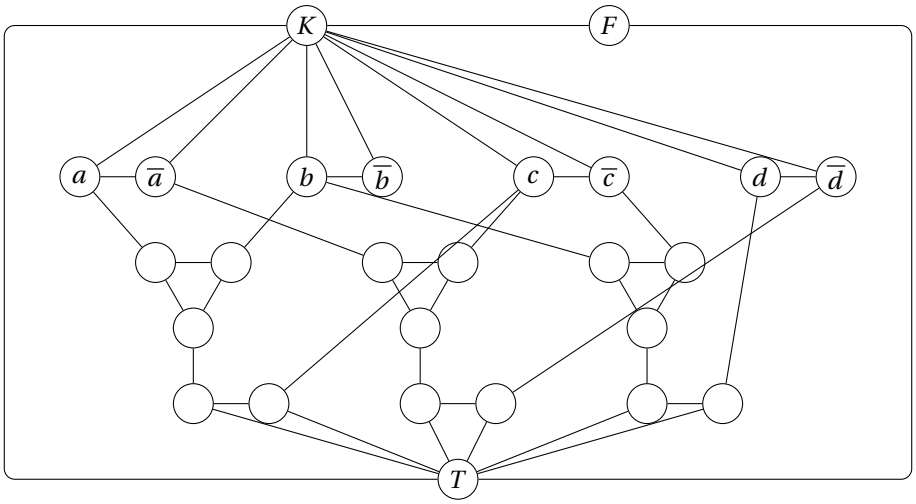


FIGURE 1 – Le graphe G_{φ_0} pour $\varphi_0 = (a \vee b \vee c) \wedge (\overline{a} \vee c \vee \overline{d}) \wedge (b \vee \overline{c} \vee d)$.

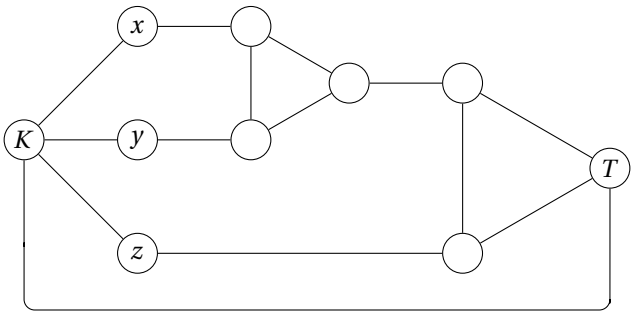


FIGURE 2 – Un gadget de clause.

Question 10 : On suppose que G_φ est 3-colorable. Montrer que φ est satisfiable. On expliquera comment construire une valuation de φ à partir d'une coloration de G_φ.
Question 11 : En déduire que `3-coloration` est NP-difficile, puis qu'il est NP-complet.
Question 12 : Montrer que pour $h \leq k$ deux entiers fixés, `h-coloration` $\leqslant^p$ `k-coloration`.
Question 13 : En déduire que pour tout $k \geq 3$, `k-coloration` est `NP-complet`.

2 Automates

L'objectif de cette partie est de proposer quelques éléments autour d'un automate, dit augmenté, construit autour d'un automate fini non déterministe et d'un sous-ensemble d'états.

Un automate fini non déterministe est un quintuplet $\mathscr{A} = (Q, \Sigma, I, \delta, F)$ avec :

- Q un ensemble fini non vide d'états, de cardinal $|Q|$;
- Σ un alphabet ;
- $I \subset Q$ l'ensemble des états initiaux ;
- $\delta : Q \times \Sigma \mapsto P(Q)$ une fonction de transition : si $q \in Q$ et $a \in \Sigma, \delta(q, a)$ désigne l'ensemble des états q' de Q tels qu'il existe une transition étiquetée par a de q vers q' ;
- $F \subset Q$ l'ensemble des états finaux

Soient $\mathscr{A} = (Q, \Sigma, I, F, \delta)$ un automate fini non déterministe et $O \subset Q$ un sous-ensemble d'états de Q. On appelle *automate augmenté* par O la paire $(\mathscr{A}, O)$, que l'on notera par la suite $\mathscr{A}_O$.

La notion de calcul est la même entre $\mathscr{A}$ et $\mathscr{A}_O$: un calcul dans $\mathscr{A}$ est une séquence d'état $q_1 \dots q_n$ de Q telle qu'il existe toujours au moins une transition entre deux états successifs de cette séquence. Le mot construit par concaténation des étiquettes de chacune des transitions est alors reconnu par l'automate.
L'ensemble O introduit la notion de *calcul réussi au seuil s.*

Soit $\mathscr{A}_O$ un automate augmenté. Un calcul passant par les états $q_i \in Q, i \in [\![0, n]\!]$ est dit *réussi au seuil* $s \in \mathbb{N}$ si :

i) $q_0 \in I$;
ii) $q_n \in F$;
iii) pour tout $i \geq 0$ tel que $i+s \leq n, \{q_i \dots q_{i+s}\} \cap O \neq \emptyset$

Soit $\mathscr{A}_O$ un automate augmenté et $s \in \mathbb{N}$. Le *langage reconnu* par $\mathscr{A}_O$ au seuil s, noté $L_s(\mathscr{A}_O)$, est l'ensemble des calculs réussis par $\mathscr{A}_O$ au seuil s.

On considère, pour les questions 14 à 16, l'automate de la figure 3, où $Q = \{q_0, q_1, q_2\}, \Sigma = \{a, b\}, I = \{q_0\}$ et $F = \{q_2\}$.

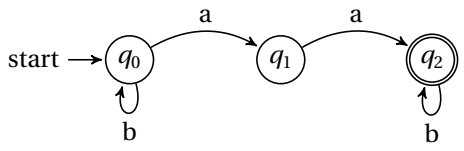


FIGURE 3 – Un exemple d'automate.

Question 14 : Donner sans justifier $L_2(\mathscr{A}_{\{q_0\}})$.
Question 15 : Donner, en justifiant votre réponse, $L_2(\mathscr{A}_{\{q_1\}})$.
Question 16 : Soit $s \in \mathbb{N}$. Donner, en justifiant votre réponse, le cardinal de $L_s(\mathscr{A}_\emptyset)$ en fonction de s où $\emptyset$ est l'ensemble vide.
Question 17 : Donner sans justification, un majorant du nombre de calculs réussis au seuil s ne passant par aucun état de O.
Question 18 : Soit maintenant un calcul réussi au seuil s qui passe par au moins un état de O. on note $\tilde{q} = q_0 \dots q_l$ la suite des états correspondants et $i_0 \dots i_k \in [\![0, l]\!]$ la suite des indices dans $\tilde{q}$ correspondant aux états de O. Donner, sans justification en fonction de s :
i) une majoration de i_0 ;
ii) un encadrement de i_k ;
iii) une majoration de $i_{j+1} - i_j$ pour $j \in [\![0, k-1]\!]$.

Question 19 : Soit $\mathscr{A}_0$ un automate augmenté, avec $|Q| = p$, où Q est l'ensemble des états de l'automate. Pour $s \in \mathbb{N}$, construire, en justifiant votre construction, un automate fini à $(s+1)p$ états reconnaissant $L_s(\mathscr{A}_O)$.

3 Mots de de Bruijn

Dans cette partie, A désigne un alphabet quelconque. Un *mot de de Bruijn* d'ordre $n \in \mathbb{N}^\star$ est un mot de $A^\star$ qui contient tous les mots de A^n une et une seule fois lorsqu'on le considère circulairement. Par exemple $abacbbcca$ est un mot de de Bruijn d'ordre 2 sur $\{a, b, c\}$ car il contient une et une seule fois chaque mot de longueur 2 de $\{a, b, c\}$, le mot aa étant obtenu en considérant que la lettre suivant le a final est la première lettre du mot, ici a.

Question 20 : Le mot $abba$ est-il un mot de de Bruijn sur $\{a, b\}$? Si oui, quel est son ordre ?
Question 21 : Quelle est la longueur d'un mot de de Bruijn en fonction de son ordre n et du nombre k de lettres dans A ?

Les questions suivantes explorent deux méthodes permettant de construire des mots de de Bruijn. La première repose sur la notion de *graphe de de Bruijn*. Le graphe de de Bruijn d'ordre n sur un alphabet A à k lettres est un graphe étiqueté oriente $B(k, n)$ tel que :
- Les sommets de $B(k, n)$ sont étiquetés par les mots de A^n.
- Les arêtes sont données par l'ensemble $\{(am, mb, b) \mid a, b \in A \text{ et } m \in A^\star\}$. Par exemple $(001, 010, 0)$ est une arête dans $B(2, 3)$ lorsque l'alphabet est $\{0, 1\}$: elle va du sommet 001 vers le sommet 010 et est étiquetée par 0.

Question 22 : Dessiner le graphe de de Bruijn $B(2, 3)$ lorsque l'alphabet est $\{0, 1\}$.
Question 23 : Montrer que le degré sortant et le degré entrant de chaque sommet dans $B(k, n)$ est égal à k.
Question 24 : En déduire le nombre d'arcs dans le graphe $B(k, n)$.

Un circuit eulérien dans un graphe orienté G est un circuit passant une et une seule fois par chaque arc de G. On admet le résultat suivant : un graphe orienté G admet un circuit eulérien si et seulement si il est connexe et pour tout sommet s de G, les degrés entrant et sortant de s sont égaux.
Question 25 : Construire $B(2, 2)$ et déterminer un circuit eulérien dans ce graphe. Constater que le mot obtenu par

concaténation des étiquettes des arêtes parcourues lors de ce circuit est un mot de de Bruijn. Quel est son ordre ?
Question 26 : Montrer qu'un circuit eulérien dans le graphe $B(k,n)$ produit un mot de de Bruijn d'ordre $n+1$ sur k symboles.
Question 27 : Montrer que pour tout alphabet A, et tout $n \geq 2$, il existe un mot de de Bruijn sur l'alphabet A d'ordre n.

L'accès à votre immeuble est protégé par un digicode. Pour entrer, il faut taper une séquence de n chiffres entre 0 et $k-1$. Le digicode fonctionne de la façon suivante : à chaque nouveau chiffre entré après le n-ème, il vérifie si les n derniers chiffres correspondent au code d'entrée. Par exemple, si $n=4$ et qu'on tape 021201, le digicode teste successivement les codes 0212, 2120 puis 1201.
Question 28 : Combien de codes faudrait-il a priori tester pour pouvoir entrer dans l'immeuble ? Si on les essaie tous successivement, combien de chiffres faudrait-il taper dans le digicode ?
Question 29 : Vu le fonctionnement du digicode, proposer une séquence astucieuse de chiffres qui garantit l'ouverture de la porte tout en diminuant le nombre total de chiffres (qu'on précisera) à entrer dans le digicode. Expliquer brièvement votre raisonnement.